

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО

Научный руководитель ОП
«Программная инженерия», профессор
департамента программной
инженерии факультета компьютерных
наук

_____ А. И. Легалов

«__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
"Программная инженерия",
старший преподаватель департамента
программной инженерии

_____ Н. А. Павлов

«__» _____ 2026 г.

**КРОСС-КОМПИЛЯТОР ЯЗЫКА ПРОГРАММИРОВАНИЯ OBERON-7 В
АСЕМБЛЕР ПРОЦЕССОРА RISC-V СИМУЛЯТОРА RARS.**

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.05.04.01 81 01-1-ЛУ

Исполнители:

Студент группы БПИНЖ232

_____ / А. А. Акулов /

«__» _____ 2026 г.

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

УТВЕРЖДЕН

RU.17701729.05.04.01 81 01-1-ЛУ

**КРОСС-КОМПИЛЯТОР ЯЗЫКА ПРОГРАММИРОВАНИЯ OBERON-7 В
АССЕМБЛЕР ПРОЦЕССОРА RISC-V СИМУЛЯТОРА RARS.**

Пояснительная записка

RU.17701729.05.04.01 81 01-1

Листов 30

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	4
1.1. Наименование программы	4
1.2. Краткая характеристика области применения программы	4
2. НАЗНАЧЕНИЕ РАЗРАБОТКИ И ОБЛАСТЬ ПРИМЕНЕНИЯ	5
2.1. Функциональное назначение	5
2.2. Эксплуатационное назначение	5
2.3. Краткая характеристика области применения программы	5
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	6
3.1. Постановка задачи на разработку программы	6
3.2. Расположение кода программы	6
3.3. Описание синтаксиса Oberon-7	6
3.3.1. Поддерживаемые типы данных	6
3.3.2. Управляющие конструкции	7
3.3.3. Арифметические операторы:	7
3.3.4. Логические операторы:	8
3.3.5. Операторы сравнения:	8
3.3.6. Специальные операции и встроенные функции	8
3.3.7. Зарезервированные ключевые слова	9
3.3.8. Форматы литералов	9
3.3.9. Модульная система и видимость символов	9
3.3.10. Формальное описание синтаксиса Oberon-7	10
3.4. Структура итогового решения	12
3.4.1. Общая архитектура компилятора	12
3.4.2. Структура проекта	13
3.4.3. Лексер	15
3.4.4. Парсер	15
3.4.5. Менеджер модулей	15
3.4.6. Семантический анализатор	16
3.4.7. Модуль генерации кода	16
3.4.7.1. Промежуточное представление (IR) и оптимизации	16
3.5. Жизненный цикл программы	17

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.5.1. Сборка программы	18
3.5.2. Запуск программы	19
3.5.2.1. Примеры использования программы в различных режимах	20
3.6. ТЕСТИРОВАНИЕ	21
3.6.1. Юнит-тесты	21
3.6.2. E2E-тесты	22
3.6.2.1. Пример E2E-теста	23
3.6.3. Непрерывная интеграция	24
3.7. Описание и обоснование выбора состава технических и программных средств	24
3.7.1. Состав технических и программных средств	24
3.7.2. Обоснование выбора состава технических и программных средств	24
4. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	26
4.1. Ориентировочная экономическая эффективность	26
4.2. Предполагаемая потребность	26
4.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными аналогами	26
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ	27
ПРИЛОЖЕНИЕ. ССЫЛКИ НА АНАЛОГИ	29

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование программы – «Кросс-компилятор языка программирования Oberon-7 в ассемблер процессора RISC-V симулятора RARS».

Наименование программы на английском языке – «Oberon-7 Programming Language Compiler to RISC-V Assembler for the RARS Simulator».

1.2. Краткая характеристика области применения программы

«Кросс-компилятор языка программирования Oberon-7 в ассемблер процессора RISC-V симулятора RARS» – приложение, которое позволяет пользователям превратить исходный код, написанный на языке программирования Oberon-7 [18], в ассемблерный код для архитектуры RISC-V [22]. Этот ассемблерный код может быть запущен в программе RARS [17], симуляторе архитектуры RISC-V. Область назначения – изучение устройства архитектуры RISC-V путем наглядной иллюстрации ассемблерного кода, в частности при изучении следующих материалов:

1. Архитектура вычислительных систем;
2. Архитектура компьютера;
3. Разработка компиляторов

Данный кросс-компилятор наглядно демонстрирует все этапы компиляции современных программ, включая семантический анализ и трансляцию в промежуточное представление, для которого можно применять самописные оптимизации машинного кода, в то же время сохраняя возможность более быстрой компиляции с отключенными данными функциями.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ РАЗРАБОТКИ И ОБЛАСТЬ ПРИМЕНЕНИЯ

2.1. Функциональное назначение

Функциональное назначение кросс-компилятора – это преобразование кода Oberon-7, языка программирования более высокого уровня, в код ассемблерный код архитектуры RISC-V.

2.2. Эксплуатационное назначение

Приложение будет применяться в образовательных целях, для демонстрации работы архитектуры RISC-V. Целевой аудиторией проекта являются люди, желающие изучить данную архитектуру, а также те, кто желает увидеть то, как в действительности работает написанный код на уровне ассемблера архитектуры RISC-V.

2.3. Краткая характеристика области применения программы

Приложение будет применяться в образовательных целях, в качестве учебного инструмента для изучения архитектуры RISC-V и процесса компилирования программ, наглядно демонстрируя основные этапы: токенизация, синтаксический анализ, семантический анализ, генерация промежуточного представления, использование машинных оптимизаций и генерация ассемблерного командной.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку программы

В результате разработки должно быть получено приложение, которое:

1. Анализирует входящий поток данных на соответствие корректному коду, написанному на языке программирования Oberon-7.
2. В случае соответствия данного кода корректной программе на языке Oberon-7, кросс-компилятор должен строить корректный ассемблерный код для архитектуры RISC-V.
3. Ассемблерный код, полученный в результате работы должен быть совместим с RARS (<https://github.com/TheThirdOne/RARS>), эмулятором архитектуры RISC-V.
4. Удобное и быстрое добавление оптимизаций, которые могут быть использованы в случае, если пользователь генерирует ассемблерный код через внутреннее представление.

3.2. Расположение кода программы

Программа расположена на сайте <https://github.com/pipitochka/o7rc>. В соответствующем README.md описаны процессы ее сборки и запуска.

3.3. Описание синтаксиса Oberon-7

3.3.1. Поддерживаемые типы данных

Тип данных / Структура	Описание
INTEGER	32-битное целое число со знаком.
BOOLEAN	Логический тип данных, принимающий значения TRUE или FALSE.
REAL	Вещественное число с плавающей точкой.
CHAR	Символьный тип данных (8 бит).
ARRAY n OF type	Массив фиксированной длины n элементов заданного типа type. Индексация начинается с 0.
RECORD ... END	Составной тип данных (структура), объединяющий набор именованных полей различных типов.
POINTER TO type	Указатель на динамически выделяемую память.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.3.2. Управляющие конструкции

Конструкция	Описание
IF ... ELSIF ... ELSE ... END	Условный оператор. Выполняется та ветвь, логическое условие которой первым вернуло TRUE.
WHILE cond DO expr END	Цикл с предусловием. Блок expr выполняется, пока логическое выражение cond истинно.
REPEAT expr UNTIL cond	Цикл с постусловием. Блок expr выполняется как минимум один раз и повторяется, пока условие cond ложно.
FOR i := start TO end BY step DO expr END	Цикл со счетчиком. Параметр шага BY step является опциональным (по умолчанию шаг равен 1).
CASE expr OF v1: e1 v2: e2 END	Оператор множественного выбора на основе значения выражения expr.

3.3.3. Арифметические операторы:

Оператор	Описание	Тип операндов
+	Сложение	Числа, множества
-	Вычитание	Числа, множества
*	Умножение / пересечение	Числа, множества
/	Деление	REAL
DIV	Целочисленное деление	INTEGER
MOD	Остаток от целочисленного деления	INTEGER

\

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.3.4. Логические операторы:

Оператор	Описание
&	Логическое И
OR	Логическое ИЛИ.
~	Логическое НЕ.

3.3.5. Операторы сравнения:

Оператор	Описание
=	Равенство.
#	Неравенство .
<	Меньше / Меньше или равно.
<=	
>	Больше / Больше или равно.
>=	

3.3.6. Специальные операции и встроенные функции

Операция / Функция	Описание
x := y	Оператор присваивания.
arr[i]	Обращение к элементу массива по индексу i.
rec.field	Обращение к полю записи.
ptr^	Явное разыменование указателя (получение значения по адресу).
NEW(ptr)	Динамическое выделение памяти в куче для типа, на который указывает ptr.
INC(x, [n]) DEC(x, [n])	Увеличение / уменьшение целочисленной переменной x на n.
ABS(x)	Возвращает абсолютное значение числа x.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.3.7. Зарезервированные ключевые слова

Категория	Ключевые слова
Структура программы	MODULE, BEGIN, END, IMPORT, RETURN
Объявления	VAR, CONST, TYPE, PROCEDURE
Управляющие конструкции	IF, THEN, ELIF, ELSE, WHILE, DO, REPEAT, UNTIL, FOR, BY, TO, CASE, OF
Типы данных	ARRAY, RECORD, POINTER TO
Специальные константы и операторы	NIL, TRUE, FALSE, DIV, MOD, OR, IN, IS

3.3.8. Форматы литералов

Тип данных	Пример записи	Описание формата
Целое число	42, -15	Стандартная десятичная запись.
Шестнадцатеричное	0FFH, 0DEADBEEFH	Шестнадцатеричная запись. Должна начинаться с цифры и заканчиваться суффиксом H.
Вещественное	3.14, 1.5E-2	Десятичная дробь. Поддерживается экспоненциальная запись.
Строка	"Hello"	Набор символов, заключенный в двойные кавычки.
Нех-строка / Символ	0DX, 41X	Шестнадцатеричный код символа, оканчивающийся суффиксом X.

3.3.9. Модульная система и видимость символов

Синтаксис	Описание и правила видимости
VAR x: INTEGER;	Приватная переменная. Доступна только внутри текущего модуля.
VAR x*: INTEGER;	Экспортируемая переменная. Маркер * делает символ доступным для импорта другими модулями.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

PROCEDURE Func*;	Экспортируемая процедура.
IMPORT Math;	Подключение внешнего модуля. Теперь можно обращаться к его экспортируемым символам.
IMPORT M := Math;	Импорт с псевдонимом (алиасом). Позволяет обращаться к модулю по короткому имени: M.Abs.

3.3.10. Формальное описание синтаксиса Oberon-7

Ниже представлено формальное описание синтаксиса языка Oberon-7 в расширенной форме Бэкуса — Наура

```

letter      = "A" | "B" | ... | "Z" | "a" | "b" | ... | "z".
digit       = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9".
hexDigit    = digit | "A" | "B" | "C" | "D" | "E" | "F".
ident       = letter {letter | digit}.
qualident   = [ident "."] ident.
identdef    = ident ["*"].
integer      = digit {digit} | digit {hexDigit} "H".
real        = digit {digit} "." {digit} [ScaleFactor].
ScaleFactor = "E" ["+" | "-"] digit {digit}.
number      = integer | real.
string      = "" {character} "" | digit {hexDigit} "X".
ConstDeclaration = identdef "=" ConstExpression.
ConstExpression = expression.
TypeDeclaration = identdef "=" type.
type         = qualident | ArrayType | RecordType | PointerType |
ProcedureType.
ArrayType    = ARRAY length {"," length} OF type.
length       = ConstExpression.
RecordType   = RECORD ["(" BaseType ")"] [FieldListSequence] END.
BaseType     = qualident.
FieldListSequence = FieldList {";" FieldList}.
FieldList    = IdentList ":" type.
IdentList    = identdef {"," identdef}.
PointerType  = POINTER TO type.
ProcedureType = PROCEDURE [FormalParameters].
VariableDeclaration = IdentList ":" type.

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

RU.17701729.05.04.01 81 01-1

```

expression      = SimpleExpression [relation SimpleExpression].
relation        = "=" | "#" | "<" | "<=" | ">" | ">=" | IN | IS.
SimpleExpression = ["+" | "-"] term {AddOperator term}.
AddOperator     = "+" | "-" | OR.
term            = factor {MulOperator factor}.
MulOperator     = "*" | "/" | DIV | MOD | "&".
factor          = number | string | NIL | TRUE | FALSE |
                  set | designator [ActualParameters]
                  | "(" expression ")" | "~" factor.
designator       = qualident {selector}.
selector        = "." ident | "[" ExpList "]" | "^" | "(" qualident ")".
set             = "{" [element {"," element}] "}".
element         = expression [".." expression].
ExpList         = expression {"," expression}.
ActualParameters = "(" [ExpList] ")" .
statement       = [assignment | ProcedureCall | IfStatement | CaseStatement |
                  WhileStatement | RepeatStatement | ForStatement].
assignment      = designator "!=" expression.
ProcedureCall   = designator [ActualParameters].
StatementSequence = statement {";" statement}.
IfStatement     = IF expression THEN StatementSequence
                  {ELSIF expression THEN StatementSequence}
                  [ELSE StatementSequence] END.
CaseStatement   = CASE expression OF case {"|" case} END.
case            = [CaseLabelList ":" StatementSequence].
CaseLabelList   = LabelRange {"," LabelRange}.
LabelRange      = label [".." label].
label           = integer | string | qualident.
WhileStatement  = WHILE expression DO StatementSequence
                  {ELSIF expression DO StatementSequence} END.
RepeatStatement = REPEAT StatementSequence UNTIL expression.
ForStatement     = FOR ident "!=" expression TO expression [BY ConstExpression]
                  DO StatementSequence END.
ProcedureDeclaration = ProcedureHeading ";" ProcedureBody ident.
ProcedureHeading    = PROCEDURE identdef [FormalParameters].
ProcedureBody       = DeclarationSequence [BEGIN StatementSequence]
                  [RETURN expression] END.
DeclarationSequence = [CONST {ConstDeclaration ";"}]

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

[TYPE {TypeDeclaration ";"}]
[VAR {VariableDeclaration ";"}]
{ProcedureDeclaration ";"}.

FormalParameters = "(" [FPSection {";" FPSection}] ")" [":" qualident].
FPSection       = [VAR] ident {";" ident} ":" FormalType.
FormalType      = {ARRAY OF} qualident.
module          = MODULE ident ";" [ImportList] DeclarationSequence
                  [BEGIN StatementSequence] END ident "." .
ImportList      = IMPORT import {";" import} ";".
import          = ident [":"=" ident].

```

3.4. Структура итогового решения

3.4.1. Общая архитектура компилятора



Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

На каждом уровне блоки являются взаимозаменяемыми и их сборка может быть отключена во время компиляции. Если же собрано с обоими вариантами, то по умолчанию используются кодогенерируемые части. При помощи команд CLI можно изменить выбранную часть, на аналогичную, которая была собрана

Наличие дублирующих реализаций обусловлено исследовательским и учебным назначением компилятора. Это позволяет наглядно сравнивать различные подходы к построению трансляторов:

1. Сравнение фронтенда: Автогенерируемые инструменты (Flex/Bison) демонстрируют классический подход на основе регулярных выражений и LALR(1)-грамматик, тогда как рукописные модули представляют рекурсивный спуск и LL(1)-анализатор, описанные создателем языка Н. Виртом [16]
2. Сравнение бэкенда: Прямая кодогенерация (AST $\rightarrow$ asm) обеспечивает максимальную скорость компиляции, но генерирует неоптимальный код. Путь через промежуточное представление (IR) позволяет применять машинно-независимые оптимизации [15].

Взаимозаменяемость любых компонентов достигается за счет строгой стандартизации интерфейсов обмена данными между слоями:

Независимо от реализации, лексер наследуется от интерфейса ITokenizer и возвращает унифицированные объекты Token. Любой парсер реализует интерфейс IParser, который принимает на вход абстрактный указатель ITokenizerPtr, ничего не зная о том, как именно были сгенерированы токены. Парсер всегда возвращает стандартизированное абстрактное синтаксическое дерево (AST).

3.4.2. Структура проекта

```

o7rc/
├─ src/
│   └─ main.cpp                # Точка входа, CLI
│   └─ tokenizer/
│       └─ ITokenizer.h        # Интерфейс лексера
│           └─ impl/
│               └─ flex/        # Flex-лексер (FlexTokenizer, oberon.l)
│               └─ hand/        # Рукописный лексер (HandTokenizer)
│               └─ debug/       # Отладочная обёртка (BufferedTokenizer)
│   └─ parser/
│       └─ IParser.h           # Интерфейс парсера
│           └─ impl/
│               └─ bison/       # Bison-парсер (BisonParser, oberon.y)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

RU.17701729.05.04.01 81 01-1

hand/	# Рукописный парсер (HandParser)
module/	# Модульная система
ModuleLoader.h	# Интерфейс загрузчика модулей
ModuleLoader.cpp	# Поиск, парсинг, кэширование модулей
sema/	# Семантический анализ
Sema.h / Sema.cpp	# Visitor по AST – проверки
SemaError.h	# Типы ошибок
util/	
Token.h	# Определение токенов
ast/	# AST-узлы (Expr, Stmt, Type, Decl, Module)
codegen/	# Прямая кодогенерация AST → RISC-V
ICodeGen.h	
RiscVCodeGen.cpp	
Emitter.cpp	
SymbolTable.cpp	
TypeInfo.cpp	
ir/	# IR-пайплайн
Value.h	# IRValue (Temp, Const, Param, Void)
Instruction.h	# IROp, IRInstr
BasicBlock.h	# Базовые блоки CFG
Function.h	# IRFunction, IRModule, IRGlobal
IRBuilder.{h,cpp}	# AST → IR
IRPrinter.{h,cpp}	# Текстовый дамп IR
IRDotExporter.{h,cpp}	# Экспорт CFG в Graphviz DOT
PassManager.{h,cpp}	# Менеджер оптимизационных проходов
RiscVIRCodeGen.{h,cpp}	# IR → RISC-V asm
passes/	
IPass.h	# Интерфейс прохода оптимизации
ConstantFolding.{h,cpp}	# Свёртка констант
CopyPropagation.{h,cpp}	# Распространение копий
DeadCodeElim.{h,cpp}	# Удаление мёртвого кода
stdlib/	# Стандартная библиотека модулей
Math.obr	# Math – стандартная библиотека
tests/	
basic.cpp	# Базовые тесты
test_factory.h	# Фабрика для параметризованных тестов
tokenizer/	# Параметризованные тесты лексера
parser/	# Параметризованные тесты парсера

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

|   └─ sema/                # Тесты семантического анализа
|   └─ ir/                  # Тесты IR, оптимизаций и памяти
|   └─ e2e/
|       └─ run_test.sh      # Скрипт запуска E2E-теста
|       └─ programs/        # 42 тестовые .obr программы (+ .in файлы)
└─ .github/workflows/ci.yml # GitHub Actions CI
└─ Dockerfile
└─ CMakeLists.txt
└─ README.md

```

3.4.3. Лексер

Модуль отвечает за чтение входного потока и трансляцию его в поток токенов. Любая встраиваемая реализация соответствует абстрактному интерфейсу `ITokenizer` с функциями `peek()` и `next()`.

1. Flex-лексер - автогенерируемый сканер на основе регулярных выражений, создающийся на этапе компиляции при помощи утилиты `Flex`
2. Рукописный лексер - рукописная реализация построенная на базе конечного автомата, с встроенной хеш-таблицей для поиска ключевых слов за $O(1)$.
3. Буферный лексер - отладочная обертка любого лексера, которая хранит весь поток токенов, а не только текущий, что помогает при дебаггинге.

3.4.4. Парсер

Модуль отвечает за синтаксический анализ потока токенов, полученного от лексера, реализованного интерфейсом `ITokenizer` и построение абстрактного синтаксического дерева в оперативной памяти. Любая встраиваемая реализация соответствует абстрактному интерфейсу `IParser`, с функцией `parse(ITokenizerPtr tz)`, возвращающей указатель на вершину построенного дерева.

1. Bison-парсер - автогенерируемый LALR(1) анализатор, создающийся на этапе компиляции при помощи утилиты `Bison`
2. Рукописный парсер - рукописная реализация построенная на методе рекурсивного спуска. Является LL(1)-анализатором, принимающим решение на основании предпросмотра следующего токена.

3.4.5. Менеджер модулей

Модуль отвечает за реализацию системы импорта внешних зависимостей.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.4.6. Семантический анализатор

Модуль отвечает за проверку семантической и логической корректности программы после того, как этап парсинга был произведен и абстрактное синтаксическое дерево было построено. Анализатор реализован на основе паттерна Visitor, который рекурсивно обходит узлы AST.

Семантический анализатор отвечает за следующие проверки:

1. Контроль областей видимости.
2. Валидация системы типов.
3. Проверка структуры подпрограмм.

3.4.7. Модуль генерации кода

Модуль отвечает за преобразование внутреннего машинезависимого представления программы (AST или IR) в целевой исполняемый код.

В зависимости от конфигурации запуска, в компиляторе используются следующие генераторы:

1. Прямой кодогенератор: Выполняет трансляцию напрямую из абстрактного синтаксического дерева (AST) в ассемблерный код. Реализован на базе паттерна Visitor.
2. IR-кодогенератор: состоит из генератора промежуточного представления (IRBuilder), отвечающего за построение графа потока управления для трёхадресного кода, менеджера оптимизаций (PassManager), отвечающего за применение оптимизаций, и генератора финального представления (RiscVIRCodeGen), отвечающего за трансляцию IR-кода в инструкции.

3.4.7.1. Промежуточное представление (IR) и оптимизации

Для реализации оптимизаций абстрактное синтаксическое дерево транслируется во внутреннее промежуточное представление. Архитектура IR в данном компиляторе базируется на трёхадресном коде и базовых блоках, образующих граф потока управления.

Структура промежуточного представления включает следующие уровни:

1. IRModule: Верхний уровень, содержащий глобальные переменные и список процедур.
2. IRFunction: Содержит локальные переменные и набор базовых блоков (BasicBlock).
3. BasicBlock: Линейная последовательность неветвящихся инструкций, заканчивающаяся терминатором (условный/безусловный переход или возврат из функции). Блоки связаны ребрами, образуя CFG.
4. IRInstr (Инструкция): Трёхадресная операция формата `dst = op src1, src2`. Включает арифметические, логические операции, операции с памятью (`load`, `store`, `alloca`) и вызовы.

Пример генерируемого трёхадресного кода

```
function Fact(n) -> i32 {
  entry:
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

%0 = alloca "n" (4 bytes)
store_local [%0], %p0
%1 = alloca "res" (4 bytes)
%2 = load [%0]
%3 = le %2, 1
br %3, bb2, bb3
bb2:
    store [%1], 1
    jmp bb1
bb3:
    ...

```

Наличие графа потока управления (CFG) позволяет применять различные машинонезависимые оптимизации с помощью компонента PassManager. На данный момент реализованы следующие:

- Constant Folding: Вычисление константных выражений на этапе компиляции.
- Copy Propagation: Замена использования скопированных переменных их исходными значениями для устранения лишних операций пересылки.
- Dead Code Elimination: Удаление недостижимых базовых блоков и инструкций, результаты которых нигде не используются.

Архитектура спроектирована таким образом, чтобы разработчик мог легко добавлять собственные проходы оптимизаций, реализуя интерфейс IPass и регистрируя новый проход в конвейере оптимизаций.

Более подробная информация находится в файле README.md в репозитории с наглядными примерами

3.5. Жизненный цикл программы

Процесс трансляции исходного кода разбит на последовательные этапы. В зависимости от переданных аргументов командной строки и опций компиляции, необходимая пользователю реализация модуля будет подключена.

1. Лексер: На основе опций компиляции и флагов, с помощью умного указателя, инстанцируется объект, реализующий интерфейс ITokenizer, который получает на вход поток данных из переданного .obj файла
2. Парсер: опций компиляции и флагов, с помощью умного указателя, инстанцируется объект, реализующий интерфейс IParser. У него вызывается функция parse(), в которую передается лексер. Функция возвращает указатель на корень абстрактного синтаксического дерева.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. Загрузка импортов: Компилятор передает полученное AST в объект ModuleLoader, который рекурсивно находит, парсит и внедряет AST всех импортированных модулей.
4. Семантический анализ: Вызывается метод analyze(), в который передается вершина AST. Происходит обход построенного дерева для выявления логических ошибок. При обнаружении ошибок компиляция прерывается.
5. Кодогенерация. На основании полученного AST происходит кодогенерация по описанным выше сценариям.

3.5.1. Сборка программы

Существует 2 варианта сборки программы: локальная сборка или на основе контейнера с всеми подключенными зависимостями.

Локальная сборка:

```
mkdir build && cd build
cmake ..
cmake --build . --parallel
```

На основе контейнера:

```
docker build -t o7rc .

docker run -it --rm -v "$(pwd)":/work o7rc

# Внутри контейнера:
mkdir build && cd build
cmake .. && cmake --build . --parallel
ctest --output-on-failure
```

В зависимости от пожеланий оператора при сборке могут быть использованы следующие cmake флаги:

Опция CMake	По умолчанию	Описание
USE_FLEX	ON	Использовать генерируемый Flex-лексер.
USE_BISON	ON	Использовать генерируемый Bison-парсер.
USE_HAND_TOKENIZER	OFF	Использовать рукописную реализацию лексера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

USE_HAND_PARSER	OFF	Использовать рукописную реализацию парсера (рекурсивный спуск).
USE_CODEGEN	ON	Включить прямую кодогенерацию ассемблера RISC-V (напрямую из AST в asm).
USE_RARS	ON	Автоматически скачать RARS/OBNC при сборке и включить E2E-тесты.
USE_DEBUG	OFF	Включить отладочный вывод токенов в консоль.

Требуется, чтобы был подключен хотя бы один из парсеров и токенайзеров для успешного завершения сборки. При отсутствии изменений флагов, их стандартные значения написаны выше.

3.5.2. Запуск программы

Запуск осуществляется как у стандартного CLI бинарного приложения

```
./o7rc input.obr -o output.asm
```

Доступные флаги CLI

Флаг / Параметр	Описание
-o <file>	Путь к выходному .asm файлу.
--tokenizer <flex hand>	Выбор лексера.
--parser <bison hand>	Выбор парсера.
--ir	Генерация кода через промежуточное представление (IR) вместо прямой трансляции AST → asm.
--opt	Включить проходы оптимизации (автоматически подразумевает использование --ir).
--dump-ir	Напечатать IR в стандартный поток ошибок stderr (до и после оптимизаций).
--dump-ir-passes	Напечатать состояние IR после каждого завершенного прохода оптимизации.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

<code>--no-sema</code>	Отключить этап семантического анализа (проверки типов, связывания имен).
<code>--dump-dot <file></code>	Экспортировать граф потока управления в формат DOT.
<code>-M <dir>, --module-path <dir></code>	Добавить каталог поиска внешних импортируемых модулей (допускается указывать флаг несколько раз).

При попытке использовать модуль, который не был скопирован, программа использует собранный. Если ни одна реализация модуля не была собрана, программа завершится с кодом 1 и соответствующим сообщением.

При нехватке информации о входном и выходном файлах, программа попросит ввести их при запуске

3.5.2.1. Примеры использования программы в различных режимах

Ниже приведены типовые сценарии запуска кросс-компилятора, демонстрирующие комбинирование различных флагов командной строки в зависимости от задач разработчика.

1. Стандартная компиляция

Прямая трансляция исходного кода в ассемблер на основе обхода AST. Выходной файл будет иметь название `program.asm`.

```
./o7rc program.obr -o program.asm
```

2. Компиляция с включенными оптимизациями

Программа сначала переводится во внутреннее промежуточное представление, после чего к ней применяются проходы оптимизации. Выходной файл будет иметь название `program.asm`.

```
./o7rc program.obr -o program.asm --ir --opt
```

3. Компиляция с включенными оптимизациями

Если `main.obr` импортирует другие модули, необходимо указать директории для их поиска с помощью флага `-M`.

```
./o7rc main.obr -o main.asm -M ./stdlib -M ../shared_libs
```

4. Выбор конкретной реализации лексера и парсера

Если `main.obr` импортирует другие модули, необходимо указать директории для их поиска с помощью флага `-M`.

```
./o7rc program.obr -o program.asm --tokenizer hand --parser hand
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. Глубокая отладка процесса оптимизации

В данном режиме в стандартный поток ошибок (stderr) будет выводиться состояние IR-кода после каждого примененного прохода оптимизации.

```
./o7rc program.obr -o program.asm --ir --opt --dump-ir-passes
```

6. Отключение семантического анализа

```
./o7rc program.obr -o program.asm --no-sema
```

Все данные флаги можно использовать одновременно.

3.6. ТЕСТИРОВАНИЕ

3.6.1. Юнит-тесты

Для юнит-тестов использована библиотека ctest

Запуск

```
ctest -E e2e --output-on-failure или ./o7rc_tests
```

Тесты запускаются для всех скомпилированных компонентов.

Юнит тесты расположены в следующих директориях и тестируют соответствующие компоненты.

Категория	Файлы	Описание
Лексер	tests/tokenizer/test_tokenizer_*.cpp	Токенизация, грамматика, сниппеты.
Парсер	tests/parser/test_parser_*.cpp	Структура AST, выражения, типы, операторы.
Sema	tests/sema/test_sema.cpp	27 тестов: позитивные + негативные.
IR Builder	tests/ir/test_ir_builder.cpp	Построение IR: модули, переменные, CFG.
IR Passes	tests/ir/test_ir_passes.cpp	ConstantFolding, CopyPropagation, DCE, PassManager.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

IR Memory tests/ir/test_ir_memory.cpp

RECORD, POINTER TO, NEW, доступ к полям, вложенные записи.

3.6.2. E2E-тесты

Требуется Java, libgc-dev и сборка с -DUSE_RARS=ON:

Запуск

```
ctest -L e2e --output-on-failure
```

Если для какого-то модуля собраны обе конфигурации, то e2e тесты будут проходить на обоих т.е. при полной сборке со всеми флагами ON каждый тест будет запускаться 8 раз (2 парсера, 2 токеназера, 2 кодогенератора)

Каждый запуск e2e теста работает по следующему сценарию

1. Компилирует .obr через OBNC → эталонный вывод
2. Компилирует .obr → .asm через o7gc → запускает в RARS → фактический вывод
3. Сравнивает выводы

Все тестовые программы:

Категория	Программы
Базовые	Hello, Arith, Assign, Negate, ConstTest
Управление	IfElse, NestedIf, CaseStmt, WhileLoop, RepeatTest, ForLoop, NestedLoop
Логика	BoolLogic, EvenOdd
Массивы	ArraySum, ArrayReverse, BubbleSort
Процедуры	Factorial, Fibonacci, MultiProc, Scope, MutualCall, RecSum
Записи / указатели	Records, RecordProc, LinkedList
Встроенные	IncDec, AbsOdd, DivMod
Алгоритмы	GCD, Power, Primes, SumDigits, Collatz
С входными данными	ReadSum, ReadMax, ReadFact, ReadGCD, ReadPower, ReadArray, ReadClassify
Модули	UseMath (импорт Math)

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Тесты с входными данными — для программ, использующих In.Int, рядом с .obr файлом лежит .in файл с входными данными. Скрипт run_test.sh автоматически подаёт его на stdin как для OBNC, так и для RARS.

3.6.2.1. Пример E2E-теста

Для демонстрации полного цикла работы компилятора (от исходного кода до выполнения на целевой архитектуре) ниже приведена цепочка действий для многомодульной программы.

1. Подготовка тестовых программ

Файл Math.obr:

```
MODULE Math;
  PROCEDURE Square*(x: INTEGER): INTEGER;
  BEGIN
    RETURN x * x
  END Square;
END Math.
```

Файл Main.obr:

```
MODULE Main;
  IMPORT Math, Out;
  VAR result: INTEGER;
  BEGIN
    result := Math.Square(5);
    Out.Int(result, 0);
    Out.Ln;
  END Main.
```

2. Компиляция исходного кода

```
./o7rc Main.obr -o Main.asm -M .
```

3. Исполнение целевого кода в симуляторе

```
java -jar rars.jar nc Main.asm./o7rc Main.obr -o Main.asm -M .
```

4. Исполнение эталонного (так как в тестах только стандартная библиотека, то ничего подключать не нужно)

```
obnc Main.obn
```

5. Сравнение результатов

Для автоматизации такого тестирования можно использовать скрипт run_test.sh.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.6.3. Непрерывная интеграция

Для обеспечения надежности и удобства разработки используется непрерывная интеграция на базе GitHub Actions. Автоматизированный процесс запускается при каждом событии `push` или `pull_request` в ветки `develop` и `main`.

Особенностью процесса тестирования является использование матричной сборки. Поскольку компилятор имеет модульную архитектуру, CI автоматически собирает и тестирует проект в 5 различных конфигурациях:

1. Только автогенерируемые инструменты (Flex + Bison).
2. Flex-лексер + Рукописный парсер.
3. Рукописный лексер + Bison-парсер.
4. Только рукописные инструменты.
5. Сборка всех модулей одновременно.

На каждой из конфигураций последовательно запускаются как Unit-тесты, так и E2E-тесты

3.7. Описание и обоснование выбора состава технических и программных средств

3.7.1. Состав технических и программных средств

Программные средства:

1. Операционная система: Разработка и тестирование проводились на macOS (15.2), а также в изолированном Docker-контейнере на базе ubuntu:24.04. Поддерживается сборка под Linux (нативно с использованием GCC/Clang) и Windows (через подсистему WSL 2, MinGW или LLVM/Clang).
2. Компилятор: Требуется компилятор с поддержкой стандарта C++17 [14] (GCC версии 9 и выше, либо Clang версии 10 и выше).
3. Система сборки: Кроссплатформенная утилита CMake [21] версии 3.20 или выше.
4. Утилиты: Flex [19] и Bison [20].
5. Среда выполнения: Java Runtime Environment версии 8 или выше для запуска симулятора RARS.

Технические средства:

1. Дисковое пространство: Не менее 200 МБ свободного места
2. Процессор с двумя ядрами и более с частотой не менее 1 ГГц.
3. 1 Гб оперативной памяти.

3.7.2. Обоснование выбора состава технических и программных средств

1. Выбор стайк обоснован тем фактом, что компилятор должен собираться независимо от ОС, а стайк предоставляет данную возможность
2. Выбор компилятора обоснован предпочтением разработчика.
3. Выбор утилит обоснован наличием их open-source реализаций

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

RU.17701729.05.04.01 81 01-1

4. Выбор JRE обоснован требованиями к запуску симулятора RARS
5. Свободное место на диске обосновано размерами собранных файлов.
6. Выбор процессора тактовой частотой от 1 ГГц обеспечивает достаточную производительность для компиляции кода, обработки лексем и построения абстрактного синтаксического дерева (AST), что является основными задачами данного компилятора
7. Выбор размера оперативной памяти: оперативной памяти объемом 1 ГБ достаточно для выполнения компилятора, обработки исходных данных и работы с абстрактным синтаксическим деревом

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Ориентировочная экономическая эффективность

В рамках данной работы расчёт экономической эффективности не предусмотрен.

4.2. Предполагаемая потребность

Данный продукт будет интересен педагогам для наглядной иллюстрации работы архитектуры RISC-V, а также обучающимся, желающим узнать то, как будет выглядеть ассемблерный код их программы, написанной на языке Oberon-7.

4.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными аналогами

Настоящий проект не преследует цели получения экономических преимуществ — он реализуется как свободное программное обеспечение и предназначен в первую очередь для использования в учебном процессе.

В данный момент прямых аналогов данному программному обеспечению на рынке не выявлено. Существуют косвенные аналоги:

1. Трансляторы в языки программирования (Восток, o7p).

Данные решения генерируют код на языках высокого уровня, требующий последующей компиляции с использованием внешних инструментов для получения исполняемого кода под архитектуру RISC-V. Это вводит избыточные шаги в процесс сборки, усложняет отладку и увеличивает общую сложность инструментальной цепочки

2. Компиляторы под операционные системы (O7/11, O7/16 и oberon-07-compiler)

Эти решения нацелены на генерацию исполняемого кода для конкретных платформ, что делает невозможным их использование на других платформах.

Таким образом, разрабатываемый компилятор является уникальным решением, обеспечивающим прямую трансляцию кода на Oberon-7 в ассемблер RISC-V, исключая избыточные этапы преобразования и не привязанную к конкретной аппаратной платформе.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
14. cplusplus – онлайн-справочник по языкам C и C++ [Электронный ресурс]. – Режим доступа: <https://en.cppreference.com/> (дата обращения: 01.01.2026).
15. Ахо Альфред В., Лам Моника С., Сети Рави, Ульман Джеффри Д., К63 Компиляторы: принципы, технологии и инструментарий, 2-е изд. : Пер. с англ. — М. : ООО “И.Д. Вильямс”, 2018.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

16. Вирт Н. Построение компиляторов/ Пер. с англ. Борисов Е. В., Чернышов Л. Н. – М.: ДМК Пресс, 2010. – 192 с.: ил.
17. TheThirdOne. RARS – RISC-V Assembler and Runtime Simulator [Электронный ресурс]. – Режим доступа: <https://github.com/TheThirdOne/RARS> (дата обращения: 01.01.2026).
18. Информационный портал OberonCore [Электронный ресурс]. – Режим доступа: <https://oberoncore.ru/> (дата обращения: 01.01.2026).
19. Levine J. Flex & Bison: Text Processing Tools. / J. Levine – O'Reilly Media, 2009. – 294 p.
20. GNU Bison – The Yacc-compatible Parser Generator [Электронный ресурс]. – Режим доступа: <https://www.gnu.org/software/bison/> (дата обращения: 30.11.2024).
21. CMake Reference Documentation [Электронный ресурс]. – Режим доступа: <https://cmake.org/cmake/help/latest/> (дата обращения: 01.01.2026).
22. RISC-V International [Электронный ресурс]. – Режим доступа: <https://riscv.org/> (дата обращения: 01.01.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ. ССЫЛКИ НА АНАЛОГИ

Приложение	Ссылка
oberon-07-compiler	https://github.com/AntKrotov/oberon-07-compiler
o7p	https://github.com/Kcchernikov/o7p
O7/11	https://github.com/valexey/Oberon-07-11-compiler/
O7/16	https://github.com/AntKrotov/oberon-07-compiler
Восток	https://github.com/Vostok-space/vostok

Дата обращения: 30.11.25.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]